Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA
(DEEMED TO BE UNIVERSITY)

THE YENEPOYA INSTITUTE OF ARTS SCIENCE COMMERCE AND MANAGEMENT
A CONSTITUENT UNIT OF YENEPOYA DEEMED TO BE UNIVERSITY
BALMATTA, MANGALORE

HIGH LEVEL DESIGN ON

Natural Language Interface for Agriculture Infrastructure

BCA AI, CLOUD COMPUTING & DEVOPS WITH IBM COMPUTER SCIENCE

SUBMITTED BY:

MOHAMMED SHAN S

23BCAICD066

25755

25755@yenepoya.edu.in

Industry Mentor: Sumit Kumar Shukla

Guided by:Deepthi

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience 3
   1.3. System overview. 4

2. System Design. 4
   2.1. Application Design 4
   2.2. Process Flow. 4
   2.3. Information Flow. 6
   2.4. Components Design 6
   2.5. Key Design Considerations 7
   2.6. API Catalogue. 7

3. Data Design. 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces 9

5. State and Session Management 11

6. Caching 11

7. Non-Functional Requirements 12
   7.1. Security Aspects 12
   7.2. Performance Aspects 12

8. References 13

1. Introduction

The Agri Assistant is an AI-powered agricultural advisory chatbot developed as part of an IBM internship project. The system provides farmers, agronomists, and agricultural students with real-time, knowledge-based guidance on crop management, pest and disease control, fertilizer recommendations, irrigation strategies, market pricing, and organic farming practices. Built using Python Streamlit framework, the application leverages a curated knowledge base derived from ICAR (Indian Council of Agricultural Research) standards and provides an interactive conversational interface.

This High Level Design (HLD) document describes the architectural approach, system components, data flow, and design considerations for the Agri Assistant platform. It is intended to provide a comprehensive technical blueprint that guides implementation, testing, and future enhancement of the system.

1.1 Scope of the Document

This document covers:

- System architecture and component design of the Agri Assistant application
- Application design including process flow, information flow, and UI/UX structure
- Data design covering the knowledge base schema, data access mechanisms, and retention policies
- API catalogue for internal service interactions and future external integrations
- Non-functional requirements including security, performance, and scalability
- Interface definitions between system components

This document does not cover low-level implementation details, database schema migrations, or deployment infrastructure provisioning scripts.

1.2 Intended Audience

This document is intended for:

- Development Team: Developers implementing and extending the Agri Assistant platform
- Project Mentors and Faculty Guides: For design review and approval
- IBM Industry Mentors: For architecture validation and technical feedback
- QA Engineers: For designing test plans based on system design

1.3 System Overview

Agri Assistant is a single-page web application deployed via Streamlit. The core system consists of a natural language query processor, a domain-specific knowledge base, utility modules for weather simulation, market pricing, and fertilizer calculation, and a session-based chat interface. The system operates in a stateless manner per conversation, with session state managed through Streamlit's built-in session management. The architecture follows a monolithic design pattern suitable for the current scope, with provisions for microservices migration in future iterations.

2. System Design

2.1 Application Design

The Agri Assistant application is structured as a single-tier web application with three logical layers:

- Presentation Layer: Streamlit-based UI with custom CSS styling, chat interface, and quick-access buttons
- Business Logic Layer: Query processor, knowledge base matcher, utility calculators (fertilizer, market prices, weather)
- Data Layer: In-memory Python dictionary serving as the knowledge base, session state management

The application runs on a Python runtime with the Streamlit framework orchestrating UI rendering and state management. All processing occurs server-side, with the browser serving as a thin client.

2.2 Process Flow

The following describes the end-to-end process flow for a user query in the Agri Assistant system:

Step	Actor / Component	Action / Description
1	User / Browser	User enters a query via chat input or clicks a quick-action button
2	Streamlit UI Layer	Input captured; session state updated with new user message
3	Query Processor	Text is normalized (lowercased, punctuation removed) and tokenized
4	Special Handler Check	System checks for price query, fertilizer calculator request, or weather request
5	Market Price Module	If crop price query detected, returns simulated market price data for crop
6	Fertilizer Calculator	If calculation requested, extracts crop and area parameters and computes NPK requirements
7	Weather Module	If weather query, returns simulated weather conditions based on current time of day
8	Knowledge Base Matcher	For general queries, performs keyword matching against DATA responses array
9	Response Generation	Matching response retrieved from knowledge base or fallback message returned
10	Streaming Output	Response streamed character-by-character to chat window for animated display
11	Session State Update	Both user message and assistant response appended to chat history

2.3 Information Flow

The information flow within Agri Assistant follows a unidirectional pipeline from user input to response output. The diagram below illustrates the data movement across system components:

USER INPUT

↓

Streamlit Session State (chat_history)

↓

process_query() → Text Normalization → Intent Detection

↓

Module Router: [Market Prices | Fertilizer Calc | Weather | Knowledge Base]

↓

Response String → Streaming Output → UI Chat Bubble

2.4 Components Design

The Agri Assistant is composed of six primary components:

Component	Responsibility	Technology
UI Layer	Chat rendering, quick buttons, styling, user input handling	Streamlit, Custom CSS
Query Processor	Text normalization, intent detection, keyword routing	Python (re, string)
Knowledge Base	Agricultural domain data storage and retrieval	Python Dictionary (DATA)
Fertilizer Calculator	NPK computation based on crop type and area	Python math functions
Market Price Module	Simulated crop pricing and trend data	Python dictionary lookup
Weather Module	Time-based simulated weather conditions	Python datetime + random
Session Manager	Conversation history and session ID management	Streamlit session_state

2.5 Key Design Considerations

- Stateless Processing: Each query is processed independently with no memory between sessions, ensuring scalability and simplicity
- Keyword-Based NLP: Pattern matching using Python regex and string operations provides deterministic responses without external AI dependencies
- Modular Architecture: Separate modules for weather, pricing, fertilizer, and knowledge base allow independent enhancement
- Streaming UX: Character-by-character response rendering provides a conversational feel with 3ms delay per character
- Knowledge Base Extensibility: The DATA dictionary structure allows easy addition of new crop types, pests, and agricultural topics
- Fallback Handling: Comprehensive fallback response guides users on supported query types when no match is found
- Session Persistence: Chat history maintained within session using Streamlit session_state for continuity

2.6. API Catalogue

The current implementation operates as a monolithic application with internal function calls rather than external REST APIs. The following table documents the internal API surface, designed for future microservices extraction:

Method	Function / Endpoint	Description	Input Parameters	Returns
CALL	process_query(text)	Main query router and response generator	text: str (user input)	str (response message)
CALL	get_weather_data()	Returns simulated weather conditions	None	tuple (condition, temp, humidity)
CALL	calculate_fertilizer(crop, area_ha)	Computes NPK fertilizer requirements	crop: str, area_ha: float	dict {urea_kg, dap_kg, mop_kg}
CALL	get_market_prices(crop)	Returns price range for a given crop	crop: str	dict {min, max, avg, trend}

Future REST API endpoints (planned for v2.0):

- POST /api/v1/query – Accept user query, return advisory response as JSON
- GET /api/v1/prices/{crop} – Fetch real-time market prices via commodity exchange integration
- GET /api/v1/weather/{location} – Fetch live weather via IMD or OpenWeatherMap API
- POST /api/v1/fertilizer/calculate – Return fertilizer recommendations based on soil test results

3. Data Design

3.1 Data Model

The primary data structure is a Python dictionary named DATA containing two keys:

- responses: A list of objects, each containing a keywords array and a response string. Keywords are matched against normalized user input to return the appropriate agricultural advisory.
- fallback: A single string providing guidance when no keyword match is found.

Field	Type	Description
DATA.responses	List[dict]	Array of topic objects with keywords and response text
responses[].keywords	List[str]	Lowercase keyword triggers for topic matching (e.g., ['rice','paddy','dhan'])
responses[].response	str	Full advisory text returned when any keyword is matched in user query
DATA.fallback	str	Default response listing supported topics when no match found
session_state.chat_history	List[dict]	In-memory list of {role, content} dicts for conversation history
session_state.session_id	str	Timestamp-based session identifier (YYYYMMDD_HHMMSS format)

3.2 Data Access Mechanism

Data access follows a read-only pattern for the knowledge base and read-write for session state:

- Knowledge Base Access: Direct Python dictionary lookup via list iteration over DATA['responses'] with O(n) complexity where n is the number of topic entries (currently 10 topics)
- Session State Access: Streamlit session_state provides key-value access with O(1) complexity for chat_history append and retrieval
- Market Price Access: O(1) dictionary lookup using crop name as key with lowercase normalization
- No database backend: All data resides in application memory, eliminating external database dependencies for v1.0

3.3 Data Retention Policies

- Chat History: Retained only for the duration of the browser session. Cleared upon page refresh, session timeout, or explicit user action via Clear Chat button
- Knowledge Base: Immutable at runtime. Changes require application code update and redeployment
- Session ID: Generated at session start, not persisted beyond session lifetime
- No PII Storage: The application does not collect, store, or transmit any personally identifiable information

3.4 Data Migration

Version 1.0 of Agri Assistant operates on an in-memory data model requiring no database migration. Future migration path for v2.0:

- Phase 1: Export DATA dictionary to JSON file for external configuration management
- Phase 2: Migrate knowledge base to a NoSQL document store (e.g., MongoDB) for dynamic content management
- Phase 3: Integrate relational database (PostgreSQL) for user session persistence, analytics, and audit logging
- Phase 4: Implement ETL pipeline for real-time market price and weather data integration via external APIs

4. Interfaces

The Agri Assistant interfaces are defined across three categories:

4.1 User Interface

- Chat Input: st.chat_input() component accepts free-text queries with Enter-key submission
- Chat Display: st.chat_message() renders conversation bubbles with role-based styling (user vs assistant)
- Quick Action Buttons: 12 predefined query buttons organized in a 4-column grid for common agricultural topics
- Clear Chat Button: Resets session chat history and triggers UI rerun
- Responsive Layout: Centered layout with 800px max-width for optimal readability on desktop and mobile

4.2 Internal Module Interfaces

- process_query(text: str) → str: Central interface accepting raw user text, returns formatted response string
- calculate_fertilizer(crop: str, area_ha: float) → dict | None: Returns fertilizer quantities or None for unsupported crops
- get_market_prices(crop: str) → dict: Returns price dictionary; for unknown crops returns default price range
- get_weather_data() → tuple[str, int, int]: Returns (condition, temperature, humidity) based on current hour

4.3 External Interfaces (Planned v2.0)

- IMD Weather API: Real-time weather data integration via India Meteorological Department API
- AGMARKNET: National Agriculture Market price data feed for live commodity prices
- Kisan Call Centre API: Integration with KCC helpline for escalation of complex queries
- SMS Gateway: Twilio or BSNL SMS gateway for offline farmer notification delivery

5. State and Session Management

Agri Assistant uses Streamlit's built-in session state mechanism for all stateful operations within a user session:

State Variable	Initialization	Purpose
chat_history	Empty list []	Stores ordered list of user/assistant message dicts for rendering and context
session_id	datetime.now().strftime(...)	Unique session identifier for potential future logging and analytics

Session lifecycle: State is initialized on first page load, persisted across re-runs within the same browser tab, and reset on tab close or explicit Clear Chat action. No cross-session persistence is implemented in v1.0.

6. Caching

Version 1.0 does not implement explicit caching as the knowledge base is small (10 topic entries) and query processing is computationally lightweight (O(n) keyword scan completing in under 1ms). The following caching strategies are planned for v2.0:

- Response Cache: LRU cache on process_query() for repeated identical queries using Python functools.lru_cache
- Market Price Cache: TTL-based cache (15-minute refresh) for external commodity price API responses
- Weather Cache: 30-minute TTL cache for IMD weather API responses to reduce external API calls
- Knowledge Base Cache: Pre-compiled regex patterns cached at application startup to improve keyword matching performance

7. Non-Functional Requirements

7.1 Security Aspects

Security Aspect	Implementation Detail
Input Sanitization	User input stripped of punctuation and lowercased via regex before processing
No Authentication	v1.0 is public-facing with no user authentication; future versions will add OAuth 2.0
No PII Collection	System does not request, store, or transmit user personal data
XSS Prevention	Streamlit framework handles HTML escaping for all user-supplied content
HTTPS Transport	Deployment on Streamlit Cloud enforces TLS 1.2+ for all connections
Dependency Security	Regular pip dependency auditing via pip-audit for known CVEs
Rate Limiting (Planned)	Request throttling per IP to prevent abuse (planned for v2.0)

7.2 Performance Aspects

Performance Metric	Target / Current State
Query Response Time	< 100ms for knowledge base lookup; streaming adds ~3ms per character
Page Load Time	< 2 seconds on standard broadband connection
Concurrent Users	Streamlit Cloud supports ~10-50 concurrent users; horizontal scaling for higher load
Knowledge Base Size	Current: 10 topics, ~15,000 characters; Target: 100+ topics with indexing
Memory Footprint	< 50MB RAM for v1.0; grows linearly with knowledge base size
Uptime Target	99.5% availability on Streamlit Cloud free tier
Mobile Performance	Responsive layout renders correctly on screens >= 320px width

8. References

#	Reference	Source / URL
1	ICAR Crop Production Guidelines	icar.org.in / KVK Regional Bulletins
2	Streamlit Documentation v1.x	docs.streamlit.io
3	Python 3.x Standard Library	docs.python.org/3
4	Pradhan Mantri Fasal Bima Yojana	pmfby.gov.in
5	AGMARKNET Commodity Prices	agmarknet.gov.in
6	IMD Agro-Meteorological Advisory	imd.gov.in/agromet
7	IBM SkillsBuild Internship Program	skillsbuild.org
8	Meghdoot Weather App for Farmers	icar.org.in/meghdoot